

GPU-Accelerated Isoalgorítmico GA+2-opt para TSP

Lucas Galdino

2025-11-22

Capítulo 1 – Introdução

- Capítulo 1 – Introdução {#cap1-introducao}
 - 1.1 Contexto e motivação {#sec-11-contexto-e-motivacao}
 - 1.2 Problema de pesquisa {#sec-12-problema-de-pesquisa}
 - 1.3 Objetivo geral {#sec-13-objetivo-geral}
 - 1.4 Objetivos específicos {#sec-14-objetivos-especificos}
 - 1.5 Justificativa {#sec-15-justificativa}
- Capítulo 2 - Revisão bibliográfica {#cap2-revisao-bibliografica}
 - 2. Fundamentação teórica e revisão bibliográfica {#sec-2-fundamentacao-e-revisao}
 - * 2.1 Otimização combinatória e o Problema do Caixeiro Viajante {#sec-21-otimizacao-tsp}
 - * 2.2 Heurísticas para o TSP {#sec-22-heurísticas-tsp}
 - 2.2.1 Entendendo o algoritmos 2-opt {#sec-221-2opt}
 - * 2.3 Algoritmos genéticos {#sec-23-algoritmos-geneticos}
 - * 2.4 Heurísticas híbridas {#sec-24-heurísticas-hibridas}
 - * 2.5 Algoritmos meméticos {#sec-25-algoritmos-memeticos}
 - 2.5.1 Algoritmo Genético + 2-opt {#sec-251-ga-2opt}
 - * 2.6 Computação em GPU e paralelização de meta-heurísticas {#sec-26-gpu-metaheurísticas}
 - 2.6.1 Taxonomia de paralelização segundo Crainic e Toulouse {#sec-261-taxonomia-paralelizacao}
 - 2.6.2 Aplicações de GPU a meta-heurísticas para o TSP {#sec-262-gpu-aplicacoes-tsp}
 - 2.6.3 Contraste arquitetural: CPU vs GPU {#sec-263-cpu-vs-gpu}
 - * 2.7 Comparação estatística de algoritmos de otimização {#sec-27-comparacao-estatistica}
 - 2.7.1 Comparações pareadas {#sec-271-comparacoes-pareadas}
 - 2.7.2 Comparações múltiplas {#sec-272-comparacoes-multiplas}

Capítulo 1 – Introdução

1.1 Contexto e motivação

O Problema do Caixeiro Viajante (Traveling Salesman Problem – TSP) é um dos problemas mais estudados em otimização combinatória (1). Tem esse nome devida a sua história clássica: um vendedor precisa visitar um conjunto de cidades, passando por cada uma exatamente uma vez, e deseja minimizar a distância total percorrida.

O objetivo é encontrar um ciclo hamiltoniano (ciclo onde deve-se passar por todos os vértices uma vez e retornar ao vértice original). Apesar de sua formulação simples, o TSP é NP-difícil (que é um problema sem solução em tempo polinomial) e, na sua formulação de decisão: “existe um tour com custo menor ou igual a B ?”, é NP-completo: não se conhece algoritmo em tempo polinomial que o resolva em geral, embora seja fácil verificar o custo de uma solução candidata. Cook (1) argumenta que essa combinação de simplicidade, dificuldade teórica e rica estrutura geométrica faz do TSP um estudo de caso central tanto para a teoria da complexidade quanto para desenvolvimento de métodos exatos e heurísticos.

Na prática, variantes do TSP aparecem em domínios como roteirização de veículos, planejamento de inspeções, manufatura e testes de circuitos, genômica e astronomia, entre outros (1). Exemplos clássicos incluem o posicionamento e ordenamento de furos em placas de circuito impresso, logística e problemas de mapeamento genético. Mesmo quando modelos reais são mais complexos (com janelas de tempo, múltiplos veículos ou restrições de capacidade), é comum validar ideias de projeto e análise de algoritmos

primeiro em instâncias clássicas do TSP, justamente pela ampla disponibilidade de referências, testes padronizados e de soluções ótimas conhecidas.

Paralelamente, a evolução do hardware trouxe processadores gráficos (GPUs) como plataforma acessível para computação de alto desempenho (HPC) (2) e com a popularização e acessibilidade, pesquisas nestas áreas cresceram rapidamente. As IAs generativas são totalmente dependentes desse tipo de hardware, por exemplo.

GPUs oferecem milhares de núcleos relativamente simples, organizados em um modelo de execução massivamente paralelo, mais próximo do paradigma SIMD/SIMT descrito por Flynn e extensões modernas (3,4). Em troca de um controle mais restrito nos fluxos de processo e memória, essas arquiteturas entregam uma grande largura de banda de memória (comunicação entre CPUs e GPUs) e uma grande taxa de operações aritméticas por segundo (FLOPs), desde que o problema ofereça operações semelhantes que possam ser executadas em paralelo, sejam elas utilizando paralelismos funcionais (mais complexos) e paralelismos de dados (mais simples). Isso torna GPUs particularmente atrativas para tarefas como o cálculo de matrizes (ganho exponencial, como demonstrado nesse trabalho), a avaliação de grandes populações de soluções e a aplicação de movimentos de vizinhança independentes em heurísticas de melhoria.

Do ponto de vista das meta-heurísticas, Crainic e Toulouse propõem uma taxonomia de paralelização que distingue, em linhas gerais, paralelismo em dados, paralelismo funcional e esquemas com múltiplas trajetórias cooperativas (5,6). De forma simplificada, pode-se falar em três tipos:

1. paralelismo *dados* (por exemplo, várias soluções de uma população sendo avaliadas em paralelo);
2. paralelismo em *tarefas* dentro de uma mesma trajetória de busca (por exemplo, vizinhanças sendo exploradas em paralelo para uma solução corrente);
3. paralelismo que mantém várias trajetórias completas de busca (estratégias de início simultâneo, modelos em ilhas, busca cooperativa).

O projeto desta monografia explora principalmente algoritmos do tipo 1, ao explorar paralelismo em dados em vizinhanças de 2-opt e na avaliação de populações em algoritmos genéticos, em linha com estudos recentes sobre heurísticas para TSP em GPU (7,8,9).

Ainda assim, nem todo algoritmo se beneficia automaticamente de uma migração para GPU. Inicializações de kernels, movimentação de dados entre CPU e GPU podem anular ganhos de paralelismo, especialmente em problemas de porte pequeno ou em implementações que realizam pouco trabalho por inicializações de kernel (10,11). Limitações de memória (VRAM) devem ser cuidadosamente calculadas para que a memória total ocupada não passe do limite do hardware. Nesses cenários, uma comparação direta entre versões em CPU e GPU exige cuidado extra: deve-se isolar o impacto da plataforma de execução sem confundir o efeito com mudanças na lógica do algoritmo. Ao longo deste trabalho, esses desafios são documentados explicitamente em estudos de caso, onde kernels são chamados de forma não cautelosa e versões otimizadas (Capítulos 3 e 4), bem como em notas técnicas específicas sobre *overhead* de lançamentos de kernel e padrões de redução em GPU.

Este trabalho insere-se nesse contexto, investigando como acelerar, utilizando GPU, um Algoritmo Genético (Genetic Algorithm – GA) híbrido com 2-opt para o TSP, de forma controlada e estatisticamente fundamentada, usando instâncias clássicas da biblioteca TSPLIB (12) e um protocolo experimental com um número adequado de repetições n_{reps} e testes estatísticos apropriados (detalhado no [Capítulo 3]). A combinação entre GA e busca local é um exemplo de algoritmo memético amplamente estudado na literatura (13,14), em que operadores evolutivos globais são complementados por heurísticas de melhoria como 2-opt (15) ou Lin–Kernighan (16). Neste Trabalho de Conclusão de Curso, usarei a nomenclatura em inglês para alguns termos técnicos como “2-opt”, “TSPLIB”, “GPU”, “CPU” e “GA”.

1.2 Problema de pesquisa

Muitos estudos em computação de alto desempenho comparam algoritmos em CPU e GPU utilizando implementações que diferem não apenas na plataforma de execução, mas também em detalhes relevantes da lógica do algoritmo (por exemplo, operadores distintos, parâmetros diferentes ou vizinhanças não equivalentes) (7,10,17). Isso torna difícil atribuir ganhos de desempenho exclusivamente ao uso da GPU, uma vez que alterações no desenho algorítmico ou na parametrização podem, por si só, explicar diferenças observadas. Neste trabalho, os parâmetros e operadores são mantidos fixos entre as variantes, e as

diferenças de desempenho observadas são discutidas à luz do protocolo experimental apresentado no [Capítulo 3].

Neste trabalho, busquei implementar as variantes do algoritmo sob as mesmas condições. A estrutura dos algoritmos é projetada para ser idêntica e otimizada tanto para CPU quanto para GPU, de forma justa, para todos os Algoritmos Genéticos e para o algoritmo de busca local 2-opt, variando apenas a forma como cada “módulo” estrutural do algoritmo é executado (em CPU ou em GPU) e o grau de paralelismo utilizado. A pergunta central pode ser formulada da seguinte forma:

Como comparar, de forma fiel, o impacto de diferentes estratégias de paralelização utilizando GPU no desempenho de um algoritmo memético (GA+2-opt) para o Problema do Caixeiro Viajante?

Responder a essa pergunta exige, ao mesmo tempo, um desenho experimental cuidadoso de hiperparâmetros e validade estatística (número significativo de instâncias, número adequado de repetições, métricas de qualidade e tempo). Os parâmetros básicos do GA — como tamanho de população n_{pop} , taxa de mutação p_{mut} , tamanho do torneio k_{trnmt} , número de iterações de 2-opt por indivíduo n_{2-opt} e número de repetições por combinação algoritmo–instância n_{reps} — são escolhidos com base em recomendações da literatura de algoritmos evolutivos (18,14) e em análises específicas documentadas nos relatórios técnicos do projeto. Esses valores e sua motivação são apresentados em detalhe no [Capítulo 3].

1.3 Objetivo geral

O objetivo geral deste trabalho é investigar e quantificar o impacto de diferentes estratégias de paralelização em processadores gráficos, com foco em:

- **Tempo de execução:** medir e comparar o tempo requerido para cada variante do algoritmo memético GA+2-opt;
- **Qualidade das soluções:** avaliar o custo final das rotas obtidas e o *gap* relativo ao ótimo conhecido, validando a viabilidade da solução;
- **Manutenção de equivalência funcional:** garantir que as quatro variantes (GA-CPU, GA-Híbrido-Ingênuo, GA-Híbrido-Otimizado, GA-FullGPU) implementem exatamente a mesma lógica algorítmica, diferindo apenas na plataforma de execução e no grau de paralelismo;
- **Metodologia estatística padronizada:** aplicar testes de hipótese apropriados (testes de normalidade, pareados e rankings) conforme recomendações da literatura proposta por Demsär (19);
- **Alinhamento com taxonomia de paralelização:** situar as estratégias adotadas no contexto da taxonomia proposta por Crainic e Toulouse (5,6).

1.4 Objetivos específicos

Para viabilizar o objetivo geral, definem-se os seguintes objetivos específicos:

1. **Implementar quatro variações isoalgorítmicas** de um Algoritmo Genético híbrido com 2-opt para o TSP, que diferem apenas na forma e no local de execução (CPU ou GPU):
 - uma versão executada puramente na CPU (**GA-CPU**), em que todas as operações (seleção, cruzamento, mutação, 2-opt e cálculo de custo) são executadas com NumPy, servindo como base em CPU para as comparações. Na prática, essa versão é utilizada como base principalmente para instâncias de menor porte (por exemplo, com $n_{coords} \leq 100$); para instâncias maiores, a comparação de tempo passa a considerar como referência a versão híbrida, conforme discutido no Capítulo 3.
 - uma versão híbrida com 2-opt executada na GPU e avaliação em CPU (**GA-Híbrido-Ingênuo[3]**), que inicia kernels **individualmente** para cada rota e transfere as soluções (ou circuitos) completos entre CPU e GPU a cada iteração;
 - uma versão híbrida otimizada (**GA-Híbrido-Otimizado**), em que 2-opt e o cálculo de custos são executados em GPU em lote (*batch*), reduzindo o número de lançamentos de kernel e o volume de dados transferidos;
 - uma versão totalmente em GPU (**GA-FullGPU**), na qual população, operadores genéticos, busca local e avaliação permanecem residentes na GPU durante toda a evolução.

Os nomes utilizados aqui seguem as implementações `GeneticAlgorithmCPU`, `GeneticAlgorithmHybridNaive`, `GeneticAlgorithmHybridOptimized` e `GeneticAlgorithmFullGPU` do framework experimental,

cujos detalhes arquiteturais são apresentados no [Capítulo 3]. [3]: Na literatura, o termo “naive” (ingênuo) é frequentemente utilizado para descrever implementações simples que não exploram otimizações avançadas.

2. **Selecionar um conjunto de instâncias da TSPLIB** com diferentes faixas de tamanho (pequenas $n_{coords} \leq 100$, médias $100 < n_{coords} \leq 400$ e grandes $n_{coords} > 400$), de forma a demonstrar os ganhos de desempenho que podem ser atingidos e garantir diversidade suficiente para analisar ganhos em escala, adotando uma divisão de faixas inspirada em estudos prévios com a TSPLIB (12), mas levemente ajustada para refletir o foco deste trabalho nas instâncias pequenas, médias e grandes selecionadas. O ajuste consiste em adotar explicitamente o número de coordenadas $n_{coords} = 100$ e $n_{coords} = 400$ como limites entre as faixas, em linha com o limiar prático utilizado para a comparação CPU/GPU e com a distribuição de tamanhos das instâncias efetivamente utilizadas nos experimentos.
3. **Definir um protocolo experimental reprodutível**, incluindo número de repetições por instância e algoritmo, critérios de parada, parâmetros do GA e limites práticos impostos pela capacidade de memória da GPU. O desenho desse protocolo segue recomendações da literatura de comparação de algoritmos e de testes estatísticos (19) e é detalhado no [Capítulo 3].
4. **Aplicar um conjunto de testes estatísticos apropriados** para comparação de algoritmos, incluindo testes de normalidade (Shapiro–Wilk), testes pareados paramétricos (teste *t-pareado*) e não paramétricos (Wilcoxon), análise de rankings em múltiplas instâncias (teste de Friedman com pós-teste de Nemenyi) e medidas de tamanho de efeito (Cohen *d*), conforme recomendações em (19) e literatura correlata de teste de hipóteses. A metodologia estatística completa é apresentada no [Capítulo 3].
5. **Analisar os resultados obtidos**, discutindo as possíveis condições em que cada variante algorítmica é vantajosa — como por exemplo, tamanho da instância n_{coords} , relação entre custos de comunicação e custo computacional, acesso à memória e volume de dados transferidos entre CPU e GPU nas direções host-to-device (H2D ou CPU->GPU) e device-to-host (D2H GPU->CPU) (2) —, limites práticos do uso de GPUs para o contexto de recursos limitados (paralelismo em uma placa gráfica já defasada e limitada) e as implicações para o projeto de algoritmos de roteamento em cenários reais, seguindo as taxonomias de paralelização de meta-heurísticas propostas por Crainic & Toulouse (5,20) e de estudos de caso em problemas de roteamento utilizando GPUs (7,8,9).

1.5 Justificativa

Do ponto de vista científico, o TSP continua sendo um problema de referência para avaliar novas ideias em heurísticas e meta-heurísticas (1). A vasta disponibilidade de estudos, instâncias utilizadas em larga escala e muitas soluções ótimas conhecidas, em particular as fornecidas pela TSPLIB (12), na qual o projeto se baseia, permite medir o desempenho de diferentes algoritmos tanto em termos de qualidade quanto em tempo de execução. Ao longo deste trabalho, será utilizada a notação n_{coords} para o número de coordenadas (cidades) de cada instância, n_{pop} para o tamanho da população do GA, n_{reps} para o número de repetições por combinação algoritmo–instância, T para tempos de execução médios e B_{H2D} , B_{D2H} para os volumes totais de dados transferidos entre CPU e GPU nas direções host-to-device e device-to-host, respectivamente. Os detalhes de como esses elementos se relacionam com os limites de memória de cada variante são discutidos no [Capítulo 3].

No campo da computação de alto desempenho, diferentes trabalhos relatam ganhos relevantes ao portar heurísticas de melhoria e algoritmos evolutivos para GPU, inclusive em variantes do TSP (7,8,9). Esses resultados indicam que GPUs são um hardware promissor para esse tipo de algoritmo, mas também evidenciam um problema recorrente: em muitas comparações, as versões em CPU e GPU diferem não apenas na plataforma de execução, mas também em detalhes de implementação e parametrização, o que dificulta isolar o efeito específico da paralelização.

Além disso, para que conclusões sobre desempenho sejam confiáveis, não basta observar poucos experimentos isolados. É necessário adotar um desenho experimental com múltiplas repetições, tanto para instâncias quanto para, medidas de dispersão e testes de hipótese adequados, como discutido na literatura de comparação de algoritmos (19). Neste trabalho, esses cuidados são incorporados ao experimento, de modo que as diferenças observadas entre variantes em CPU e GPU possam ser atribuídas, com maior segurança, às estratégias de paralelização avaliadas.

Este trabalho justifica-se por combinar três elementos que, em conjunto, fortalecem a contribuição científica:

1. **Implementação de heurísticas clássicas em GPU** (particularmente 2-opt e operadores do AG), explorando o paralelismo de forma explícita.
2. **Comparação entre variantes funcionalmente equivalentes do algoritmo em CPU e GPU**, mantendo fixos operadores, parâmetros e critérios de parada, de modo a isolar o efeito da plataforma de execução. [4] [4]: Algumas variações dentro deste contexto são esperadas, mas de modo geral, o algoritmo segue a mesma estrutura. Sempre há a possibilidade de haver pequenas variações, mas o autor busca manter, ao máximo, a consistência entre as variantes.
3. **Aplicação de metodologia estatística rigorosa**, com múltiplas repetições por instância, testes de hipótese adequados e medidas de tamanho de efeito.

Os capítulos seguintes discutem em que condições cada variante é vantajosa, quais os limites práticos do uso de GPUs e recursos limitados e como os resultados dialogam com estudos prévios de heurísticas em GPU.

[!note] ainda verei se usarei ou não essa nota

1.6 Organização do trabalho

Este texto está organizado da seguinte forma:

- **Capítulo 2 – Fundamentação Teórica / Revisão Bibliográfica:** apresenta os conceitos básicos de otimização combinatória e do TSP, revisa heurísticas de construção e melhoria (com ênfase em 2-opt), discute algoritmos genéticos e meta-heurísticas híbridas, introduz princípios de computação em GPU e paralelização de meta-heurísticas, e resume abordagens estatísticas para comparação de algoritmos.
- **Capítulo 3 – Materiais e Métodos:** descreve o ambiente computacional, a arquitetura do framework desenvolvido (incluindo abstrações de backend e estratégias isoalgorítmicas), os detalhes do Algoritmo Genético híbrido com 2-opt nas quatro variantes consideradas, o conjunto de instâncias TSPLIB utilizado, os parâmetros experimentais e o protocolo estatístico adotado.
- **Capítulo 4 – Resultados:** apresenta os resultados numéricos dos experimentos, incluindo estatísticas por instância, análises agregadas por faixa de tamanho, medidas de speedup e qualidade das soluções, além da interpretação dos testes estatísticos aplicados.
- **Capítulo 5 – Discussão:** interpreta criticamente os resultados à luz da literatura, discute as implicações dos achados para o projeto de algoritmos de roteamento em GPU e analisa limitações do estudo.
- **Capítulo 6 – Conclusões e Trabalhos Futuros:** sintetiza as principais contribuições do trabalho, responde explicitamente aos objetivos propostos e indica possíveis extensões, como a aplicação do framework a problemas de roteirização mais complexos.
- Elementos pós-textuais, como referências, glossário, apêndices técnicos e anexos com tabelas completas de resultados, são apresentados ao final do documento, conforme normas da instituição.

Capítulo 2 - Revisão bibliográfica

2. Fundamentação teórica e revisão bibliográfica

Este capítulo apresenta conceitos teóricos e a bibliografia utilizada para contextualizar o problema estudado e as escolhas metodológicas adotadas. São discutidos o Problema do Caixeiro Viajante (TSP) no contexto da otimização combinatória (1,21), sua importância geral e exemplos de utilização. Também são apresentadas generalizações do problema e sua relação com problemas de roteamento de veículos e variantes correlatas (22,23,24), heurísticas e meta-heurísticas clássicas para o TSP (15,16,25), aprofundando-se então no tópico de algoritmos genéticos e algoritmos meméticos (13,14), princípios de computação em GPU (2,7,8), taxonomias de paralelização de meta-heurísticas (5,6,20) e de comparação estatística de algoritmos de otimização (19).

2.1 Otimização combinatória e o Problema do Caixeiro Viajante

O TSP pode ser formulado, na versão simétrica¹, como um problema de encontrar um ciclo hamiltoniano de custo mínimo — podendo este ser a distância percorrida, combustível gasto, ou mesmo uma combinação de fatores. Para fins deste trabalho, o custo será também chamado de distância — em um grafo completo não direcionado $G = (V, E)$, no qual cada vértice em V representa uma cidade e cada aresta $(i, j) \in E$ possui um custo $c_{ij} \geq 0$ entre as cidades i e j (1). O objetivo é determinar uma combinação das cidades que minimize a soma total dos custos, retornando à cidade de origem. Geralmente em aplicações práticas assume-se que a matriz de custos é métrica e euclidiana, isto é, os custos derivam de distâncias euclidianas entre coordenadas no plano.

Do ponto de vista da complexidade, o TSP é um problema NP-difícil em sua forma de otimização e NP-completo em sua forma de decisão (1). Isso significa que, em geral, não se conhece um algoritmo em tempo polinomial que resolva instâncias arbitrárias em grande escala. Mesmo que seja relativamente fácil verificar o custo de uma ou várias soluções candidatas, não podemos afirmar que esta é solução a exata em tempo polinomial. Algoritmos modernos como **Held-Karp** (26) com uma complexidade temporal $O(n^2 2^n)$ e o algoritmo **Concorde** (1) com uma complexidade temporal indefinida, conseguem resolver instâncias com milhares de cidades², mas seu tempo de execução cresce exponencialmente (ainda melhor que a força bruta $(n-1)!$ com o tamanho do problema, tornando-os impraticáveis para instâncias muito grandes.

Ao longo das últimas décadas, o TSP consolidou-se também como um padrão de avaliação empírica de algoritmos, graças à disponibilidade de coleções de instâncias padronizadas, como a **TSPLIB** (12). Essas coleções incluem instâncias com diferentes tamanhos, estruturas e origens (geográficas, sintéticas, industriais), a maioria delas com soluções ótimas conhecidas e obtidas por métodos exatos. No presente trabalho, são utilizadas algumas dessas instâncias como base para avaliar e comparar heurísticas e variantes paralelas de um algoritmo memético (combinações de algoritmos evolutivos com busca local) (13,9).

2.2 Heurísticas para o TSP

Devido à dificuldade de resolver instâncias grandes do TSP exatamente, uma vasta literatura de heurísticas tem sido desenvolvida para produzir boas soluções em tempos de computação aceitáveis (1). De forma geral, heurísticas podem ser agrupadas em dois grandes tipos: heurísticas de construção e heurísticas de melhoria.

Heurísticas de construção produzem uma solução viável “do zero”, frequentemente seguindo regras simples, também algoritmos gulosos, como escolher iterativamente o vizinho mais próximo ou inserir cidades em posições que causem o menor aumento de custo. Exemplos incluem a heurística do vizinho mais próximo (nearest neighbor — NN), heurísticas de inserção (insertion heuristics) e variantes baseadas em árvores de extensão mínimas (Minimum Spanning Trees — MSTs), como o Algoritmo de Christofides. Embora rápidas e fáceis de implementar, essas estratégias tendem a gerar soluções de qualidade moderada, servindo sobretudo como ponto de partida para métodos mais sofisticados. Resultados clássicos da literatura mostram que, em instâncias métricas, heurísticas baseadas em MST e em Christofides podem oferecer garantias de aproximação sobre o custo ótimo de $\frac{3}{2}B^*$, aspecto discutido em textos de referência em logística e roteamento (27).

Heurísticas de melhoria partem de uma solução inicial e aplicam sucessivos movimentos locais que procurem reduzir seu custo. Entre essas, as chamadas k -opt são particularmente influentes: um movimento 2-opt consiste em remover duas arestas de um circuito e reconectar os segmentos resultantes, escolhendo a reconexão que produz um circuito ainda viável e de menor comprimento (15). Em instâncias euclidianas é comum interpretar o 2-opt como um mecanismo para eliminar cruzamentos, mas, genericamente, ele também pode melhorar circuitos que não apresentam interseções evidentes, ao substituir pares de arestas por combinações de menor custo (15). Movimentos 3-opt e extensões mais complexas, como o algoritmo

¹Na versão assimétrica do TSP, os custos de viagem entre pares de cidades podem diferir dependendo da direção (ou seja, $c_{ij} \neq c_{ji}$). Em TSPs simétricos, o custo do cálculo de distâncias pode ser representado por uma matriz triangular que armazena apenas uma das metades (superior ou inferior) sem a diagonal, o que reduz o número de entradas únicas de n^2 para $n(n-1)/2$.

²O algoritmo Held-Karp “troca” a complexidade temporal da força bruta $O(n!)$ por uma complexidade espacial $O(n2^n)$, tornando sua viabilidade limitada a ~ 40 problemas. Já o algoritmo Concorde, baseado em técnicas de ramificação e corte, é capaz de resolver instâncias com até 85.900 cidades, mas seu desempenho depende fortemente da estrutura específica da instância (1).

de Lin–Kernighan, generalizam essa ideia (16). Estas e outras heurísticas de melhoria desempenham papel central em muitos algoritmos modernos para TSP, tanto como procedimentos isolados quanto como componentes de meta-heurísticas, sendo utilizadas como procedimentos de busca local.

Neste trabalho, a heurística 2-opt é utilizada como busca local básica acoplada ao Algoritmo Genético, em linha com estudos que combinam heurísticas de construção simples com procedimentos de melhoria mais complexas para obter soluções de alta qualidade em tempo razoável (25,13,9).

2.2.1 Entendendo o algoritmos 2-opt A formulação básica de um movimento 2-opt pode ser descrita da seguinte forma (15): dado um circuito $C = (v_0, v_1, \dots, v_{n-1}, v_0)$ e dois índices i e j tais que $0 \leq i < j < n - 1$, considera-se a remoção das arestas (v_i, v_{i+1}) e (v_j, v_{j+1}) e a inserção das arestas (v_i, v_j) e (v_{i+1}, v_{j+1}) . A variação de custo associada ao movimento é dada por

$$\Delta C = (c_{i,j} + c_{i+1,j+1}) - (c_{i,i+1} + c_{j,j+1}),$$

onde $c_{i,j}$ denota o custo (ou distância) entre as cidades i e j . Um movimento 2-opt é aceitável quando $\Delta C < 0$, isto é, quando a substituição das arestas reduz o comprimento total do circuito (15).

Em implementações práticas, percorrem-se pares de índices (i, j) em alguma ordem predefinida até que nenhum movimento que gere um custo menor seja encontrado ou até que um **limite máximo de iterações** seja atingido. O conceito de “iteração” no contexto de 2-opt refere-se a uma passagem completa sobre a vizinhança do circuito: em cada iteração, examina-se um conjunto de pares (i, j) e aplicam-se todos os movimentos que reduzem o custo.

Quando determinamos “10 iterações em 2-opt”, significa que o algoritmo realizará até 10 dessas passagens completas, parando antes se não houver mais melhorias possíveis. Sem um limite máximo, o algoritmo continuaria até atingir um ótimo local — o que pode ser custoso computacionalmente em circuitos grandes (16). No contexto de algoritmos meméticos (Seção 2.5), o número de iterações de 2-opt por indivíduo é um parâmetro crucial que equilibra intensidade da busca local com custo computacional: muitas iterações podem refinar excessivamente cada solução — o alto custo computacional despendido segue a lei dos rendimentos decrescentes —, enquanto poucas podem deixar melhorias óbvias sem explorar.

De forma mais detalhada, o algoritmo 2-opt clássico pode ser descrito como um procedimento iterativo de busca local aplicada a um único circuito. Em linhas gerais, o algoritmo segue os passos:

1. Começar com um circuito viável inicial C (obtido por uma heurística de construção qualquer).
2. Definir uma ordem de varredura para pares de índices (i, j) com $0 \leq i < j < n - 1$. No presente trabalho, utiliza-se a ordem lexicográfica padrão, em que para cada i , varia-se $j = i + 2$ até $n - 1$ (a restrição $j \geq i + 2$ evita movimentos triviais [5] e garante que o segmento a ser revertido tenha comprimento mínimo (15)). [5]: Movimentos triviais são aqueles que não alteram efetivamente o circuito, como tentar reverter um seguimento de i a $i + 1$, pois $c_{i,i+1} = c_{i+1,i}$ é uma aresta única e sua reversão não muda o circuito, no caso do TSP clássico explorado nest documento. O mesmo não se aplica para o ATSP.
3. Para cada par (i, j) , calcular ΔC conforme a expressão acima.
4. Se $\Delta C < 0$, aplicar o movimento 2-opt correspondente, o que equivale a reverter o segmento $(v_{i+1}, \dots, v_j)$ do circuito, obtendo um novo circuito C' , e marcar que houve melhora.
5. Repetir o processo de varredura enquanto forem encontrados movimentos com $\Delta C < 0$ (isto é, até atingir um ótimo local em relação à vizinhança 2-opt) ou até que um número máximo de iterações seja alcançado.

Em uma implementação ingênua, a vizinhança 2-opt de um circuito com n cidades possui ordem $O(n^2)$ movimentos possíveis, o que implica um custo potencialmente elevado quando todos os pares (i, j) são examinados de forma exaustiva. Diversas otimizações são discutidas na literatura, como o uso de listas de vizinhança, estruturas de dados para poda de movimentos claramente não promissores e estratégias de parada antecipada (16,13). No contexto desta monografia, o foco recai principalmente na forma como esse procedimento é paralelizado e acoplado ao Algoritmo Genético, mais do que em otimizações finas da vizinhança em CPU.

flowchart TD

```
Start(["Início: Circuito C"]) --> Init["Iteração ← 0<br/>melhorou ← verdadeiro"]
Init --> CheckIter{"iteração < max<br/>E melhorou?"}
```

```

CheckIter -->|Não| End(["Fim: Retorna C"])
CheckIter -->|Sim| ResetFlag["melhorou ← falso<br/>iteração ← interação + 1"]
ResetFlag --> LoopI["Para i = 0 até n-3"]
LoopI --> LoopJ["Para j = i+2 até n-1"]
LoopJ --> CalcDelta["Calcular ΔC = (c_ij + c_{i+1,j+1}) - (c_{i,i+1} + c_{j,j+1})"]
CalcDelta --> CheckDelta{"ΔC < 0?"}
CheckDelta -->|Sim| Apply["Reverter segmento (i+1..j)<br/>melhorou ← verdadeiro"]
CheckDelta -->|Não| NextJ
Apply --> NextJ["Próximo j"]
NextJ -->|Mais j| LoopJ
NextJ -->|Fim j| NextI["Próximo i"]
NextI -->|Mais i| LoopI
NextI -->|Fim i| CheckIter

style Start fill:#e1f5e1
style End fill:#ffe1e1
style CheckDelta fill:#fff4e1
style Apply fill:#e1f0ff

```

Figura 2.1: Fluxograma do algoritmo 2-opt clássico. O laço externo continua enquanto houver melhorias e o número de iterações não exceder o limite. Em cada iteração, todos os pares (i, j) são examinados; movimentos com $\Delta C < 0$ são aplicados imediatamente, revertendo o segmento do circuito.

[!caution] missing pseudocode

2.3 Algoritmos genéticos

Algoritmos Genéticos (Genetic Algorithms – GAs) são meta-heurísticas inspiradas em princípios de evolução biológica, nas quais uma população de soluções candidatas é iterativamente modificada por operadores análogos à seleção natural, recombinação e mutação (14,18). Na sua forma mais simples, um AG mantém uma população de indivíduos representando soluções para o problema; em cada geração, indivíduos são selecionados com base em uma função de aptidão (fitness), recombinados por operadores de cruzamento e perturbados por operadores de mutação. A nova população resultante substitui total ou parcialmente a anterior, e o processo se repete até que um critério de parada seja satisfeito.

No contexto do TSP, é comum representar cada solução como uma permutação das cidades, utilizar operadores de cruzamento especializados para rotas — como Order Crossover (OX), Partially Mapped Crossover (PMX), entre outros — e empregar mutações que preservem a viabilidade da permutação, como trocas de posição entre duas cidades (13). Estudos clássicos mostram que GAs podem produzir soluções competitivas para o TSP quando combinados com operadores e parâmetros adequados (14).

Do ponto de vista formal, um GA opera sobre uma população $P(t) = \{x_1^{(t)}, x_2^{(t)}, \dots, x_{n_{pop}}^{(t)}\}$ de indivíduos (soluções candidatas) na geração t . Cada indivíduo x_i possui um valor de aptidão (fitness) $f(x_i)$ que mede a qualidade da solução; no caso do TSP, $f(x_i)$ corresponde ao custo do circuito representado por x_i , e busca-se minimizar esse valor. A cada geração, o GA aplica três operadores principais (14,18):

1. **Seleção:** escolhe indivíduos de $P(t)$ com probabilidade proporcional (ou baseada em ranking/torneio) à sua aptidão, favorecendo soluções de melhor qualidade. Formalmente, a probabilidade de selecionar x_i pode ser dada por $p_i = f(x_i) / \sum_{j=1}^{n_{pop}} f(x_j)$ (seleção proporcional) ou por esquemas de torneio, nos quais um subconjunto aleatório de indivíduos compete e o melhor é escolhido.
2. **Cruzamento (recombinação):** combina pares de indivíduos selecionados (“pais”) para produzir descendentes. No TSP, operadores como OX e PMX preservam a viabilidade das permutações, herdando subsequências de um dos pais e preenchendo as posições restantes com a ordem do outro (13). A taxa de cruzamento p_c controla a fração de pares submetidos a esse operador.
3. **Mutação:** introduz pequenas perturbações aleatórias em indivíduos, mantendo diversidade na população. No TSP, mutações típicas incluem troca de duas cidades ou reversão de segmentos. A taxa de mutação p_{mut} controla a probabilidade de cada indivíduo sofrer mutação.

Após aplicar esses operadores, forma-se a nova população $P(t+1)$ por meio de um esquema de substituição

(geracional, com elitismo, steady-state, etc.). O processo se repete até que um critério de parada seja satisfeito (número máximo de gerações, convergência, custo-alvo, etc.). A Figura 2.2 ilustra o fluxo geral de um GA.

```

BEGIN AGA
  Make initial population at random.
  WHILE NOT stop DO
    BEGIN
      Select parents from the population.
      Produce children from the selected parents.
      Mutate the individuals.
      Extend the population adding the children to it.
      Reduce the extend population.
    END
    Output the best individual found.
  END AGA

flowchart TD
  Start(["Início"]) --> InitPop["Gerar população inicial P(0)<br/>aleatória ou heurística"]
  InitPop --> Eval0["Avaliar aptidão f(x_i)<br/>para cada indivíduo"]
  Eval0 --> SetT["t ← 0"]
  SetT --> CheckStop{"Critério de<br/>parada<br/>satisfeito?"}
  CheckStop -->|Sim| Output["Fim: Retorna<br/>melhor solução"]
  CheckStop -->|Não| Selection["Seleção: escolher pais<br/>de P(t) com base em f(x_i)"]
  Selection --> Crossover["Cruzamento: combinar pares<br/>de pais (taxa p_c) gerando<br/>descendentes"]
  Crossover --> Mutation["Mutações: perturbar indivíduos<br/>(taxa p_mut) para diversidade"]
  Mutation --> EvalNew["Avaliar aptidão dos<br/>novos indivíduos"]
  EvalNew --> Replace["Substituição: formar P(t+1)<br/>combinando P(t) e descendentes<br/>(com ou sem elitismo)"]
  Replace --> IncT["t ← t + 1"]
  IncT --> CheckStop

```

Figura 2.2: Fluxograma de um Algoritmo Genético (GA) clássico. A população evolui iterativamente por meio de seleção, cruzamento e mutação, até que um critério de parada seja atingido (14,18).

2.4 Heurísticas híbridas

De forma ampla, heurísticas híbridas combinam duas ou mais estratégias de busca, procurando explorar sinergias entre métodos com forças complementares. No contexto de problemas de roteamento e, em particular, do TSP, é comum combinar heurísticas de construção (como vizinho mais próximo, inserções ou heurísticas baseadas em MST) com heurísticas de melhoria (como movimentos k -opt) ou com meta-heurísticas populacionais (como GAs), de modo que uma componente forneça boas soluções iniciais e outra detalhe a exploração de vizinhanças (25).

Diversos autores classificam heurísticas híbridas em categorias como: (i) **hibridização em nível de solução**, na qual uma meta-heurística utiliza sistematicamente uma busca local (ou outro procedimento) para melhorar indivíduos candidatos; (ii) **hibridização em nível de algoritmo**, em que diferentes técnicas (por exemplo, um método exato e um heurístico) são combinadas em fases distintas ou em esquemas de cooperação; e (iii) **hibridizações multiestágio**, nas quais diferentes heurísticas atuam em momentos distintos do processo de resolução (inicialização, intensificação, diversificação, etc.) (13,25). Essas categorias ajudam a organizar o espectro de abordagens híbridas reportadas na literatura, embora nem sempre haja consenso absoluto sobre as fronteiras entre elas.

Uma extensão importante dessa ideia é o conceito de algoritmos meméticos, nos quais operadores evolutivos (seleção, cruzamento, mutação) são combinados com heurísticas de busca local aplicadas a indivíduos da população, de forma sistemática (13). Em problemas de roteamento, isso resulta em esquemas GA+LS, nos quais um GA guia a exploração global do espaço de soluções e um procedimento de melhoria, como 2-opt ou Lin-Kernighan, refina soluções promissoras. Esses algoritmos tendem a oferecer melhor equilíbrio entre exploração e intensificação do que GAs “puros”, ao custo de maior tempo computacional por iteração.

Trabalhos como o de Fujimoto e Tsutsui (9) exploram justamente essa combinação GA+busca local para o TSP em ambientes de computação paralela, motivando a adoção de uma abordagem semelhante neste

estudo. A próxima subseção aprofunda o conceito de algoritmos meméticos e a subseção 2.5.1 descreve, em linhas gerais, o esquema GA+2-opt adotado nesta monografia.

2.5 Algoritmos meméticos

Algoritmos meméticos podem ser vistos como uma família de meta-heurísticas evolutivas que incorporam, explicitamente, operadores de busca local ao ciclo de evolução populacional (13). A metáfora usual associa o termo “meme” a unidades de informação que se propagam e se transformam, de modo análogo a genes, mas em um nível mais alto de organização: além da recombinação e mutação de indivíduos, há um processo de “aprendizado” local que modifica soluções de forma dirigida.

Em um algoritmo memético típico, a cada geração, parte dos indivíduos (por exemplo, os mais aptos ou uma amostra da população) é submetida a uma busca local, que pode ser uma heurística de melhoria como 2-opt, 3-opt ou Lin–Kernighan no caso do TSP. Esse procedimento permite refinar soluções já boas, acelerando a convergência em direção a ótimos locais de alta qualidade. O GA fornece a diversidade global por meio de operadores de cruzamento e mutação, enquanto a busca local intensifica a exploração em torno de regiões promissoras do espaço de soluções.

O desenho de um algoritmo memético envolve decisões importantes, como: (i) **quais indivíduos** serão submetidos à busca local (apenas a elite, um subconjunto aleatório, toda a população); (ii) **com que frequência** a busca local será aplicada (toda geração, gerações alternadas, fases específicas do processo); e (iii) **com que intensidade** cada chamada de busca local será executada (por exemplo, número máximo de iterações de 2-opt). Essas escolhas impactam o balanço entre qualidade das soluções e custo computacional, bem como a diversidade mantida na população.

Revisões como a de Larrañaga et al. (13) destacam que algoritmos meméticos para o TSP figuram entre as abordagens mais competitivas, especialmente quando combinam operadores de cruzamento adequados ao TSP, mutações que preservam a viabilidade das rotas e procedimentos de melhoria eficientes. No presente trabalho, o algoritmo memético considerado é construído a partir de um **GA clássico** — entendido aqui como um GA com seleção por torneio, cruzamento Order Crossover (OX), mutação por troca de posições (*swap*) e substituição geracional com elitismo (14,18) — acoplado a uma rotina de 2-opt que refina localmente os indivíduos após os operadores genéticos.

Formalmente, seja $P(t)$ a população na geração t , e seja $\mathcal{L}(x, k)$ a aplicação de k iterações de 2-opt a um indivíduo x . No esquema memético adotado, após gerar os descendentes por cruzamento e mutação, aplica-se $\mathcal{L}(x_i, n_{2-opt})$ a alguns (ou todos) os novos indivíduos, de modo que a população $P(t+1)$ contenha soluções já refinadas. O parâmetro n_{2-opt} controla a intensidade da busca local: valores maiores aumentam a qualidade média das soluções, mas elevam o custo computacional por geração. Essa combinação de exploração global (GA) e intensificação local (2-opt) caracteriza o algoritmo memético e é o núcleo do presente estudo, conforme detalhado a seguir.

2.5.1 Algoritmo Genético + 2-opt O esquema GA+2-opt adotado neste trabalho segue a linha de algoritmos meméticos clássicos para o TSP (13,25), nos quais um Algoritmo Genético opera sobre uma população de permutações de cidades e, em momentos específicos do ciclo evolutivo, aplica-se uma rotina de busca local 2-opt a alguns indivíduos. Em alto nível, cada iteração (geração) do algoritmo pode ser descrita pelos seguintes passos:

1. **Inicialização** ($t = 0$): gerar uma população inicial $P(0)$ de rotas, por exemplo, a partir de permutações aleatórias ou de heurísticas de construção simples (vizinho mais próximo, inserções).
2. **Avaliação**: calcular o custo (comprimento) $f(x_i)$ de cada rota $x_i \in P(t)$, que servirá como medida de aptidão.
3. **Seleção**: escolher indivíduos para reprodução com base em seus valores de aptidão, utilizando seleção por torneio de tamanho k_{trnmt} , favorecendo soluções de menor custo.
4. **Cruzamento**: combinar pares de indivíduos selecionados por meio de Order Crossover (OX), produzindo descendentes que preservam a estrutura de permutação e herdam subsequências de ambos os pais.
5. **Mutação**: aplicar perturbações leves às rotas (trocas de posição de cidades, com taxa p_{mut}) para manter diversidade genética.
6. **Busca local 2-opt**: para cada descendente (ou um subconjunto, conforme a estratégia), aplicar $\mathcal{L}(x, n_{2-opt})$ — isto é, executar até n_{2-opt} iterações de 2-opt — de modo a refinar localmente as

rotas resultantes. Esta etapa é o que transforma o GA em um algoritmo memético, introduzindo intensificação explícita da busca.

7. **Substituição:** formar a nova população $P(t+1)$ combinando os melhores indivíduos de $P(t)$ (elitismo) com os descendentes refinados, garantindo que as melhores soluções sejam preservadas ao longo das gerações.

O processo repete-se até que um critério de parada seja satisfeito (número máximo de gerações, estagnação da aptidão ou atingimento do custo ótimo conhecido). A Figura 2.3 ilustra o fluxo completo do GA+2-opt memético.

flowchart TD

```

Start(["Início"]) --> Init["Gerar população inicial P(0)<br/>(permutações aleatórias ou NN)"]
Init --> Eval0["Avaliar f(x_i) para cada<br/>indivíduo em P(0)"]
Eval0 --> SetT["t ← 0"]
SetT --> CheckStop{"Critério de<br/>parada?"}
CheckStop -->|Sim| Output(["Fim: Retorna melhor solução"])
CheckStop -->|Não| Selection["Seleção por torneio<br/>(k_trnmt) de pais"]
Selection --> Crossover["Cruzamento (OX)<br/>gerar descendentes"]
Crossover --> Mutation["Mutaç o (swap)<br/>taxa p_mut"]
Mutation --> LocalSearch["Busca Local: aplicar<br/>n_2opt itera  es de 2-opt<br/>a cada descendente"]
LocalSearch --> EvalNew["Avaliar f(x_i) dos<br/>descendentes refinados"]
EvalNew --> Replace["Substitui  o: formar P(t+1)<br/>com elitismo (melhores de P(t)<br/>+ descendentes)"]
Replace --> IncT["t ← t + 1"]
IncT --> CheckStop

```

style Start fill:#e1f5e1
 style Output fill:#ffe1e1
 style LocalSearch fill:#ffcdd2
 style Crossover fill:#e1f5fe
 style Mutation fill:#ffe1f0

Figura 2.3: Fluxograma do algoritmo memético GA+2-opt. A busca local de 2-opt (etapa destacada) é aplicada após os operadores genéticos, refinando cada descendente antes da formação da nova população. Esse acoplamento caracteriza o algoritmo como memético, combinando exploração global (GA) e intensificação local (2-opt) (13,25).

No contexto deste trabalho, essa etapa de busca local será posteriormente mapeada para diferentes implementações em CPU e GPU (GA-CPU, GA-Híbrido-Ingênuo, GA-Híbrido-Otimizado, GA-FullGPU), mantendo a mesma lógica de aplicação de 2-opt (mesma vizinhança, mesmos limites de iterações n_{2-opt}), de forma a preservar o caráter isoalgorítmico entre as variantes. Os detalhes operacionais, incluindo a forma precisa de parametrizar n_{2-opt} , n_{pop} , p_{mut} e os critérios de parada do GA, são apresentados no Capítulo 3.

2.6 Computação em GPU e paralelização de meta-heurísticas

Processadores gráficos (GPUs) evoluíram, nas últimas décadas, de dispositivos voltados principalmente para renderização gráfica para plataformas de computação de uso geral (GPGPU), amplamente utilizadas em aplicações científicas e de inteligência artificial (2). O modelo de programação CUDA, por exemplo, organiza o trabalho em grades (*grids*) de blocos de threads, seguindo um paradigma de execução massivamente paralelo próximo ao SIMT (Single Instruction, Multiple Threads), relacionado às classificações de arquiteturas de Flynn (3,4). Nessa configuração, milhares de threads executam o mesmo kernel sobre dados distintos, o que é adequado a tarefas com alto grau de paralelismo em dados.

2.6.1 Taxonomia de paralelização segundo Crainic e Toulouse É importante distinguir **estratégias de paralelização** (que dizem respeito a *como* o trabalho computacional é distribuído entre processadores ou threads) de **estratégias de hibridização** (discutidas na Seção 2.4, que tratam de *combinar* diferentes métodos de busca). Crainic e Toulouse propuseram uma taxonomia para paralelização de meta-heurísticas que as distingue em três grandes tipos (5,6):

- **Tipo 1 (paralelismo em dados):** avaliações de soluções, cálculos de custo e procedimentos de busca local são distribuídos entre vários processadores ou threads, explorando principalmente o para-

lelismo inerente aos dados (múltiplas soluções avaliadas simultaneamente, múltiplos movimentos de vizinhança testados em paralelo). GPUs são particularmente adequadas a esse tipo de paralelismo devido ao seu grande número de threads.

- **Tipo 2 (paralelismo em tarefas ou funcional):** diferentes partes ou fases de um mesmo algoritmo (por exemplo, avaliação, seleção, aplicação de vizinhanças) são executadas em paralelo ou em pipelines, caracterizando paralelismo funcional. Isso pode envolver, por exemplo, diferentes estágios de um pipeline executando simultaneamente, ou decomposições por domínio em que partes do espaço de busca são exploradas independentemente.
- **Tipo 3 (esquemas cooperativos):** múltiplas execuções independentes de uma meta-heurística interagem por meio de mecanismos de cooperação, como modelos em ilhas (populações distribuídas trocando indivíduos periodicamente), execuções multi-start com troca de informação, ou memórias compartilhadas entre processos de busca. Este tipo envolve coordenação entre trajetórias de busca.

No contexto deste trabalho, as quatro variantes do GA+2-opt exploram principalmente **paralelismo do tipo 1**: a aplicação de 2-opt a múltiplos indivíduos (ou múltiplos movimentos por indivíduo) é distribuída entre threads da GPU, e a avaliação de rotas também se beneficia de paralelismo em dados. As versões híbridas (GA-Híbrido-Ingênuo e GA-Híbrido-Otimizado) também apresentam elementos de **tipo 2**, ao dividir o trabalho entre CPU e GPU em fases distintas — por exemplo, operadores genéticos (seleção, cruzamento, mutação) executados na CPU, enquanto a busca local 2-opt é delegada à GPU. Esquemas de **tipo 3** — como modelos em ilhas com múltiplas populações cooperativas — não são abordados diretamente nesta monografia, embora constituam uma direção natural de extensão futura.

2.6.2 Aplicações de GPU a meta-heurísticas para o TSP Aplicações de GPU a meta-heurísticas para o TSP exploram principalmente paralelismo em dados (tipo 1), seja na avaliação em massa de rotas em algoritmos genéticos (por exemplo, via cálculos de redução em GPU), seja aplicando paralelismo a movimentos de vizinhança em heurísticas de melhoria (7,8,9). Outros trabalhos investigam paralelização de Recozimento Simulado, Colônias de Formigas (ACO) e outras meta-heurísticas em GPU, geralmente aproveitando o grande número de threads para explorar múltiplas soluções ou vizinhanças em cada passo da busca (28,29,30).

Esses estudos motivam o uso de GPUs como plataforma para acelerar algoritmos meméticos, mas também evidenciam desafios relacionados à movimentação de dados entre CPU e GPU, à escolha de granularidade adequada de kernels, à ocupação dos multiprocessadores e às limitações de memória (VRAM). Esses aspectos são retomados no Capítulo 3, ao descrever o desenho dos kernels de 2-opt e as estratégias de controle de memória adotadas neste trabalho.

2.6.3 Contraste arquitetural: CPU vs GPU As Figuras 2.4, 2.5 e 2.6 ilustram diferenças fundamentais entre arquiteturas de CPU e GPU. A Figura 2.4 apresenta um contraste conceitual de alto nível: CPUs possuem poucos núcleos complexos, com grande quantidade de silício dedicada a controle de fluxo, predição de desvios e caches de múltiplos níveis, favorecendo execução rápida de código sequencial com ramificações complexas. GPUs, por outro lado, dedicam a maior parte do die a unidades aritméticas simples (ALUs) organizadas em centenas ou milhares de núcleos especializados, sacrificando controle complexo em favor de throughput massivo quando muitas threads executam operações similares sobre dados distintos. Essa diferença arquitetural é central para entender a adequação de GPUs a algoritmos que expõem paralelismo em dados, como avaliação de populações e busca local 2-opt paralela.

A Figura 2.5 detalha essa organização interna: enquanto CPUs investem em hierarquias de cache profundas e controle avançado de execução fora de ordem, GPUs organizam-se em múltiplos streaming multiprocessors (SMs), cada um com dezenas de CUDA cores compartilhando memória local (shared memory) e registradores, permitindo milhares de threads ativas simultaneamente. Essa arquitetura favorece workloads com alta densidade aritmética e padrões de acesso regular à memória.

A Figura 2.6 ilustra a organização interna de uma GPU moderna: a hierarquia começa na memória global (DRAM de alta largura de banda), passa por memórias compartilhadas locais a cada SM, e chega aos registradores de cada thread. A execução ocorre em warps (grupos de 32 threads) que executam a mesma instrução simultaneamente (modelo SIMT). Essa estrutura impõe restrições — divergências de fluxo dentro de um warp causam serialização, e padrões de acesso irregulares à memória global reduzem eficiência — mas oferece throughput excepcional quando essas condições são atendidas.

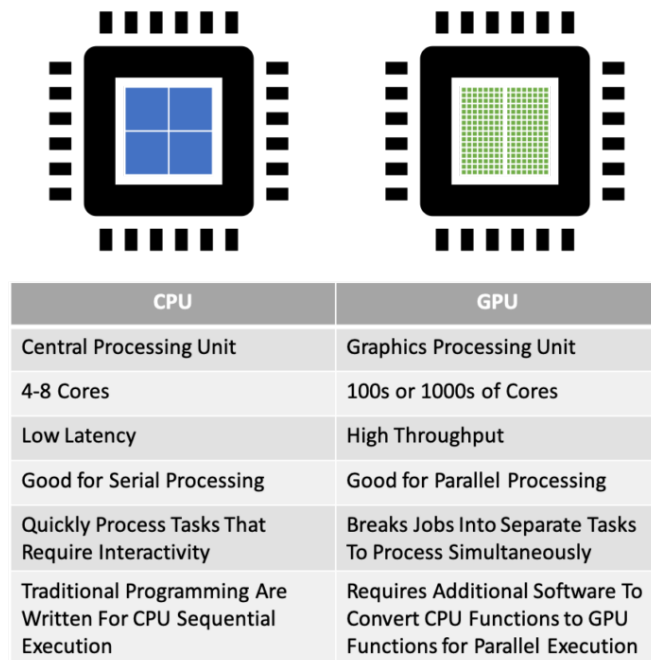


Figura 1: Contraste conceitual entre arquiteturas de CPU e GPU. CPUs possuem poucos núcleos complexos com vasta área dedicada a controle e cache, otimizando código sequencial com ramificações. GPUs dedicam a maior parte do die a unidades aritméticas (ALUs) simples, maximizando throughput para workloads com paralelismo em dados. Fonte: LayerStack Blog.

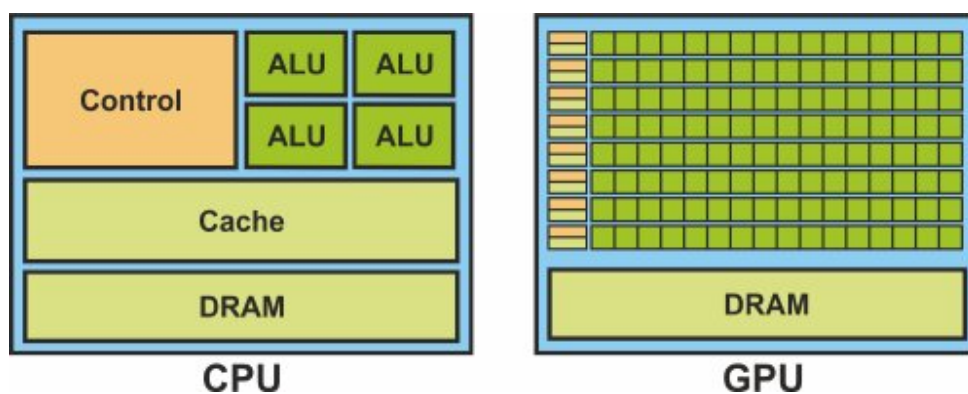


Figura 2: Comparação detalhada de alocação de silício entre CPU e GPU. A CPU dedica grande parte do die a controle (control) e cache, com poucos núcleos (ALU) complexos. A GPU inverte essa proporção, alocando a maior parte do espaço a unidades aritméticas e DRAM, com hierarquia de memória mais simples. Essa organização torna GPUs ideais para workloads que expõem paralelismo massivo em dados (31).

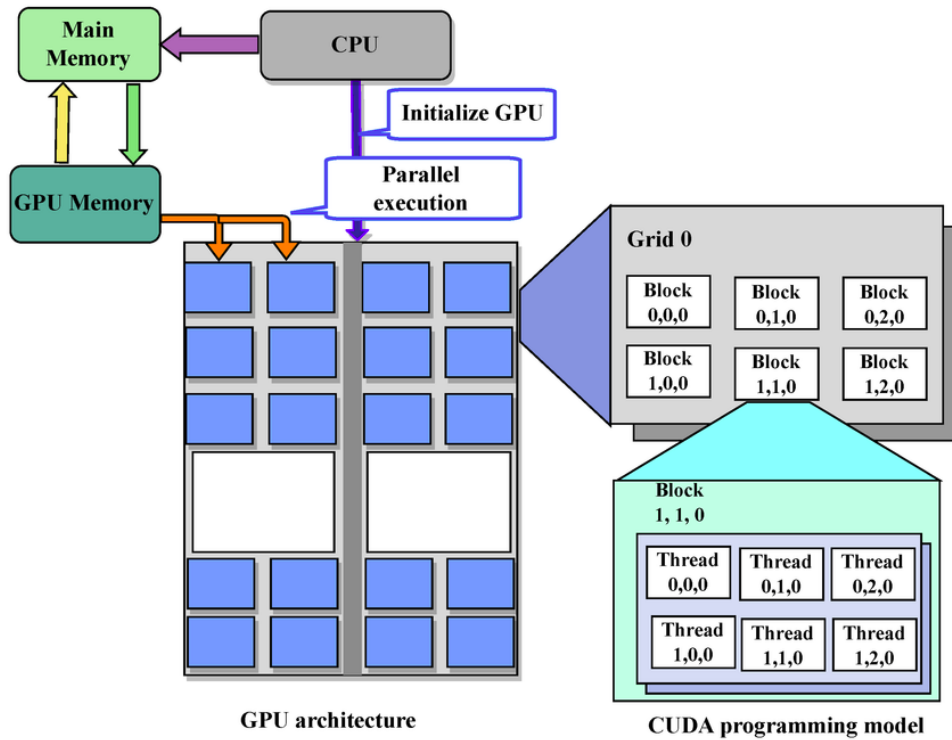


Figura 3: Organização interna de uma GPU moderna. A hierarquia de memória vai da DRAM global até memórias compartilhadas por SM e registradores por thread. Threads são agrupadas em blocos alocados a SMs, e executam em warps de 32 threads sob modelo SIMT. Essas características guiam o design de kernels CUDA eficientes para 2-opt paralelo (32).

Essas três figuras estabelecem o contexto arquitetural que orienta as decisões de design dos kernels CUDA apresentados no Capítulo 3: explorar paralelismo em dados (tipo 1) mapeando threads a indivíduos ou movimentos 2-opt, minimizar divergência de fluxo controlando a granularidade de paralelização, e gerenciar cuidadosamente transferências entre memória global e shared memory para maximizar throughput.

2.7 Comparação estatística de algoritmos de otimização

Meta-heurísticas estocásticas, como GAs e algoritmos meméticos, produzem resultados que variam de execução para execução devido ao uso de aleatoriedade em diversos pontos (inicialização, seleção, mutação, entre outros). Por esse motivo, a comparação de algoritmos de otimização não pode se basear em uma única execução por instância, tampouco apenas em médias simples sem análise de variabilidade. A literatura de comparação de algoritmos enfatiza a importância de utilizar múltiplas instâncias de teste, múltiplas repetições por combinação algoritmo–instância e métricas que considerem tanto qualidade da solução quanto tempo de execução (19).

Demšar (19) discute procedimentos estatísticos apropriados para comparar algoritmos em vários problemas, recomendando o uso de testes de hipótese pareados (como o teste *t-pareado* ou o teste de Wilcoxon) quando se deseja comparar dois algoritmos em uma coleção de instâncias, e testes baseados em rankings, como o teste de Friedman seguido de pós-testes de Nemenyi, quando mais de dois algoritmos são avaliados simultaneamente. Medidas de tamanho de efeito, como o *d* de Cohen, complementam a análise ao quantificar a magnitude prática das diferenças observadas.

Neste trabalho, esses princípios gerais orientam o desenho experimental e a análise dos resultados apresentados nos capítulos seguintes, garantindo que as comparações entre variantes em CPU e GPU sejam realizadas de forma estatisticamente fundamentada. De maneira mais concreta, para cada combinação algoritmo–instância (4 algoritmos $\times$ 38 instâncias), são executadas múltiplas repetições independentes, registrando-se, no mínimo, medidas de qualidade da solução (custo final ou *gap* em relação ao melhor valor conhecido) e de tempo de execução.

2.7.1 Comparações pareadas Para comparações **par a par** entre duas variantes (por exemplo, GA-CPU versus uma versão em GPU) ao longo de várias instâncias, adota-se uma rotina em duas etapas: (i) verificação de normalidade das distribuições de interesse por meio do teste de Shapiro–Wilk (33) e (ii) aplicação de um teste pareado apropriado. Quando a hipótese de normalidade é considerada aceitável, utiliza-se o teste *t-pareado* de Student (34); caso contrário, recorre-se ao teste não paramétrico de Wilcoxon para amostras pareadas (35). Em ambos os casos, medidas de tamanho de efeito, como o d de Cohen (36), são utilizadas para qualificar a relevância prática das diferenças detectadas. Valores típicos de d de Cohen são interpretados como: $|d| < 0.2$ (efeito negligenciável), $0.2 \leq |d| < 0.5$ (efeito pequeno), $0.5 \leq |d| < 0.8$ (efeito médio), e $|d| \geq 0.8$ (efeito grande).

2.7.2 Comparações múltiplas Quando três ou mais algoritmos são comparados simultaneamente em um conjunto de instâncias (como ocorre com as quatro variantes isoalgorítmicas GA-CPU, GA-Híbrido-Ingênuo, GA-Híbrido-Otimizado e GA-FullGPU), emprega-se o teste de Friedman (37) sobre os rankings médios dos algoritmos em cada problema, seguido de pós-testes de Nemenyi (38) quando apropriado, conforme as recomendações de Demšar (19). Esse procedimento permite identificar, com controle de erro tipo I, quais pares de algoritmos apresentam diferenças estatisticamente significativas em termos de desempenho médio. O teste de Friedman verifica a hipótese nula de que todos os algoritmos têm desempenho equivalente, enquanto o pós-teste de Nemenyi ajusta os valores-p para múltiplas comparações, reduzindo o risco de falsos positivos.

Os detalhes específicos de parametrização desses testes, bem como o conjunto exato de métricas analisadas (tempo, qualidade, *speedup*, entre outras), são apresentados no Capítulo 3 ao descrever o protocolo experimental, e retomados no Capítulo 4 ao discutir os resultados obtidos.

Referências

- [1] COOK, W. J. **In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation.** [s.l.] Princeton University Press, 2012.
- [2] NVIDIA CORPORATION. **CUDA C++ Best Practices Guide.** [s.l.: s.n.].
- [3] FLYNN, M. J. Some Computer Organizations and Their Effectiveness. **IEEE Transactions on Computers**, v. C-21, n. 9, p. 948–960, 1972.
- [4] DRAFT. **DRAFT: almasi2002high.**, 2025.
- [5] CRAINIC, T. G.; TOULOUSE, M. Parallel Strategies for Meta-Heuristics. Em: GLOVER, F.; KOCHENBERGER, G. A. (Eds.). **Handbook of Metaheuristics.** Boston, MA: Springer, 2003. p. 475–513.
- [6] CRAINIC, T. G.; TOULOUSE, M. Parallel meta-heuristics. Em: **Handbook of metaheuristics.** [s.l.] Springer, 2010. p. 497–541.
- [7] SCHULZ, C. et al. GPU computing in discrete optimization. Part II: Survey focused on routing problems. **EURO journal on transportation and logistics**, v. 2, n. 1-2, p. 159–186, 2013.
- [8] ROCKI, K.; SUDA, R. **High Performance GPU Accelerated Local Optimization in TSP.** Proceedings of the 2013 IEEE 27th International Symposium on Parallel and Distributed Processing Workshops and PhD Forum. **Anais...** IPDPSW '13. Washington, DC, USA: IEEE Computer Society, 2013. Disponível em: <<http://dx.doi.org/10.1109/IPDPSW.2013.227>>
- [9] FUJIMOTO, N.; TSUTSUI, S. **A highly-parallel tsp solver for a gpu computing platform.** Numerical Methods and Applications: 7th International Conference, NMA 2010, Borovets, Bulgaria, August 20-24, 2010. Revised Papers 7. **Anais...** Springer, 2011.
- [10] VAN LUONG, T.; MELAB, N.; TALBI, E.-G. GPU Computing for Parallel Local Search Metaheuristic Algorithms. **IEEE Transactions on Computers**, v. 62, n. 1, p. 173–185, 2013.

- [11] LUONG, T. V.; MELAB, N.; TALBI, E.-G. **GPU-based island model for evolutionary algorithms**. Proceedings of the 15th annual conference on Genetic and evolutionary computation. **Anais...**2013.
- [12] REINELT, G. TSPLIB—A Traveling Salesman Problem Library. **ORSA Journal on Computing**, v. 3, n. 4, p. 376–384, 1991.
- [13] LARRAÑAGA, P. et al. Genetic algorithms for the travelling salesman problem: A review of representations and operators. **Artificial Intelligence Review**, v. 13, n. 2, p. 129–170, 1999.
- [14] GOLDBERG, D. E. **Genetic Algorithms in Search, Optimization, and Machine Learning**. [s.l.] Addison-Wesley, 1989.
- [15] CROES, G. A. A method for solving traveling-salesman problems. **Operations Research**, v. 6, n. 6, p. 791–812, 1958.
- [16] LIN, S.; KERNIGHAN, B. W. An effective heuristic algorithm for the traveling-salesman problem. **Operations Research**, v. 21, n. 2, p. 498–516, 1973.
- [17] BENAINI, A.; BERRAJAA, A. **Genetic algorithm for large dynamic vehicle routing problem on GPU**. 2018 4th International Conference on Logistics Operations Management (GOL). **Anais...IEEE**, 2018.
- [18] EIBEN, A. E.; SMITH, J. E. **Introduction to Evolutionary Computing**. 2nd. ed. [s.l.] Springer, 2015.
- [19] DEMŠAR, J. Statistical Comparisons of Classifiers over Multiple Data Sets. **J. Mach. Learn. Res.**, v. 7, p. 1–30, dez. 2006.
- [20] ALBA, E. **Parallel metaheuristics: a new class of algorithms**. [s.l.] John Wiley & Sons, 2005.
- [21] ROSENKRANTZ, D. J.; STEARNS, R. E.; LEWIS, P. M. An Analysis of Several Heuristics for the Traveling Salesman Problem. **SIAM J. Comput.**, v. 6, p. 563–581, set. 1977.
- [22] RAFF, S. Routing and scheduling of vehicles and crews: The state of the art. **Computers & Operations Research**, v. 10, n. 2, p. 63–211, 1983.
- [23] BODIN, L. et al. The state of the art in the routing and scheduling of vehicles and crews. 1981.
- [24] TAN, S.-Y.; YEH, W.-C. The vehicle routing problem: State-of-the-art classification and review. **Applied Sciences**, v. 11, n. 21, p. 10295, 2021.
- [25] VOUDOURIS, C.; TSANG, E. Guided local search and its application to the traveling salesman problem. **European Journal of Operational Research**, v. 113, n. 2, p. 469–499, 1999.
- [26] HELD, M.; KARP, R. M. **A dynamic programming approach to sequencing problems**. Proceedings of the 1961 16th ACM National Meeting. **Anais...**: ACM '61. New York, NY, USA: Association for Computing Machinery, 1961. Disponível em: <<https://doi.org/10.1145/800029.808532>>
- [27] SIMCHI-LEVI, D. et al. The logic of logistics. **Theory, algorithms, and applications for logistics and supply chain management**, 2005.
- [28] BINJUBIER, M. et al. A GPU Accelerated Parallel Genetic Algorithm for the Traveling Salesman Problem. **Journal of Soft Computing and Data Mining**, v. 5, n. 2, p. 137–150, 2024.

- [29] REY, A. et al. **A CPU-GPU parallel ant colony optimization solver for the vehicle routing problem.** Applications of Evolutionary Computation: 21st International Conference, EvoApplications 2018, Parma, Italy, April 4-6, 2018, Proceedings 21. **Anais...**Springer, 2018.
- [30] ABDELATTI, M.; SODHI, M. **An improved GPU-accelerated heuristic technique applied to the capacitated vehicle routing problem.** jun. 2020.
- [31] PAZ-GALLARDO, A.; PLAZA, A. **A New Morphological Anomaly Detection Algorithm for Hyperspectral Images and its GPU Implementation.** Proceedings of SPIE - The International Society for Optical Engineering. **Anais...**set. 2011.
- [32] SHAH, N. et al. CUDA-Optimized GPU Acceleration of 3GPP 3D Channel Model Simulations for 5G Network Planning. **Electronics**, v. 12, p. 3214, jul. 2023.
- [33] SHAPIRO, S. S.; WILK, M. B. An analysis of variance test for normality (complete samples). **Biometrika**, v. 52, n. 3-4, p. 591–611, 1965.
- [34] STUDENT. The probable error of a mean. **Biometrika**, v. 6, n. 1, p. 1–25, 1908.
- [35] WILCOXON, F. Individual comparisons by ranking methods. **Biometrics Bulletin**, v. 1, n. 6, p. 80–83, 1945.
- [36] COHEN, J. **Statistical Power Analysis for the Behavioral Sciences.** 2nd. ed. New York: Routledge, 1988.
- [37] FRIEDMAN, M. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. **Journal of the American Statistical Association**, v. 32, n. 200, p. 675–701, 1937.
- [38] NEMENYI, P. B. **Distribution-Free Multiple Comparisons.** tese de doutorado—[s.l.] Princeton University, 1963.